

Defending the Governed Data Product for AI

Eight raw banking files, made into data an AI assistant can query correctly — and only for the clients each user may see

Berka (PKDD'99) take-home · technical walkthrough · November 2026

One story in five parts



1 The problem

Natural-language questions on bank data

2 The principle

Treat the AI as a dumb and dangerous client

3 Three proofs

Trustworthy data · one meaning · access it can't bypass

4 The evidence

10 questions, measured — and its limits

5 Honesty & next

What I'd challenge, what I learned, what's next

The thread through all five

Traceable

every number comes from a command you can re-run

No silent repairs

nothing is fixed or dropped without a record

Never trusted blindly

the AI gets no chance to be quietly wrong

Filter in the engine

where the model's text cannot reach it

The problem: an answer that looks right and isn't



Task 1 · Data quality

Rules with a severity — and a reason for every severity



Task 2 · AI-ready product

A model, a semantic layer, and an LLM answering 10 questions



Task 3 · Entitlements

Each user sees only their clients; the LLM cannot bypass it

Three things must be true — all three

- The data is clean — a duplicate payment inflates every total
- Every word means one thing — 'a bad loan' cannot mean two things
- Every user sees only their own clients — a bank may not show everyone everything

Miss one, and the assistant gives an answer that looks right and is wrong — or leaks.

1,079,680
rows in 8 source files

10
business questions

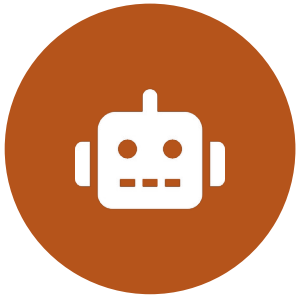
5,369
clients, 4,500 accounts

1–2 days
expected effort (more on that later)

The principle: treat the AI as a dumb and dangerous client

Dumb — it guesses

- A column that sounds right
- The wrong date or quarter
- A count where a share was asked
- Its own meaning of 'problematic'



Dangerous — its SQL runs

- A second statement smuggled in
- Internal schemas, huge results
- Trying to see more clients
- Anything it writes hits customer data

So every layer removes a reason to guess, or a way to do harm:

Meaning

Semantic layer

removes the reasons to guess

Shape

SQL guard

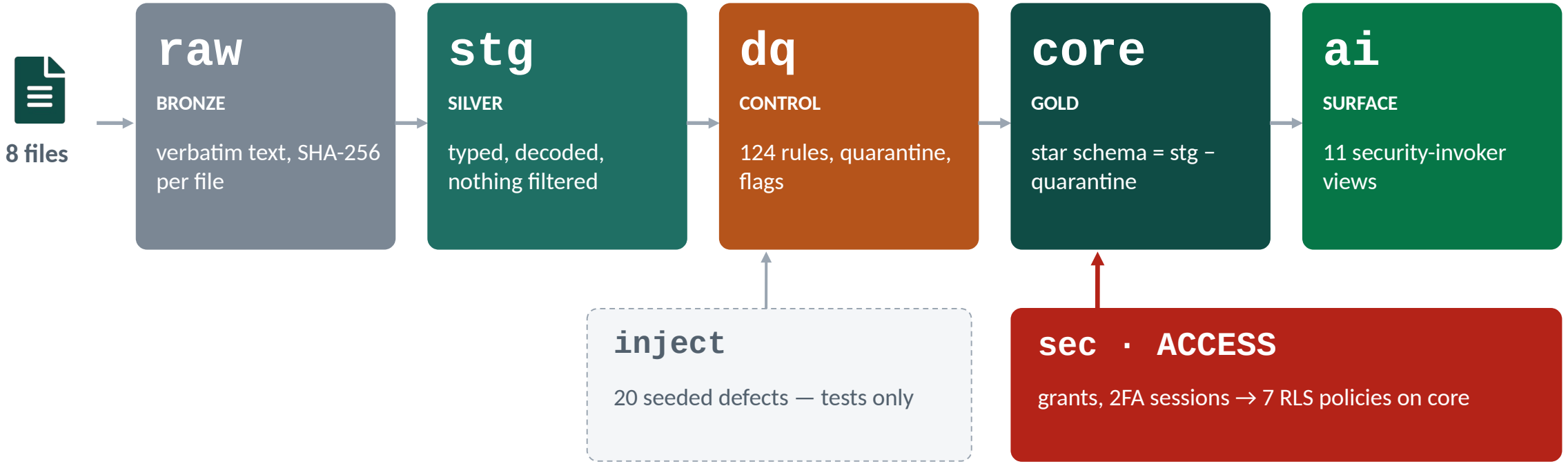
one read-only SELECT, bounded

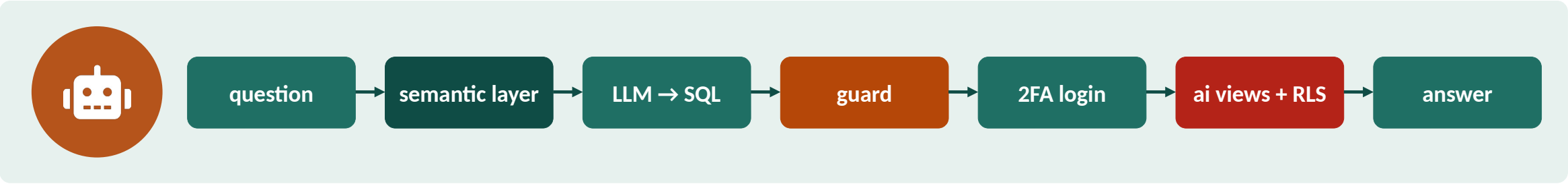
Rows

Row level security

the only security boundary

The map: six schemas in one PostgreSQL database





Proof 1 · Trustworthy data, nothing silently changed

raw | stg | dq | core | ai | sec

you are here

8 files
landed verbatim

→

21 cleaning steps
only 1 removes a row

→

124 rules
19 fire · 14 defect types

→

1 quarantined
440 warn flags

BLOCK

Quarantined with a reason; excluded, reversible

1 row

WARN

Kept; the flag travels to the answer as a caveat

440 flags

INFO

Standardised or logged; no answer changes

12 rules

Same check, opposite decision

14 zero-amount postings are real interest runs → info (blocking would break 14 balance chains to fix nothing). A negative amount would be block: money moves the wrong way.

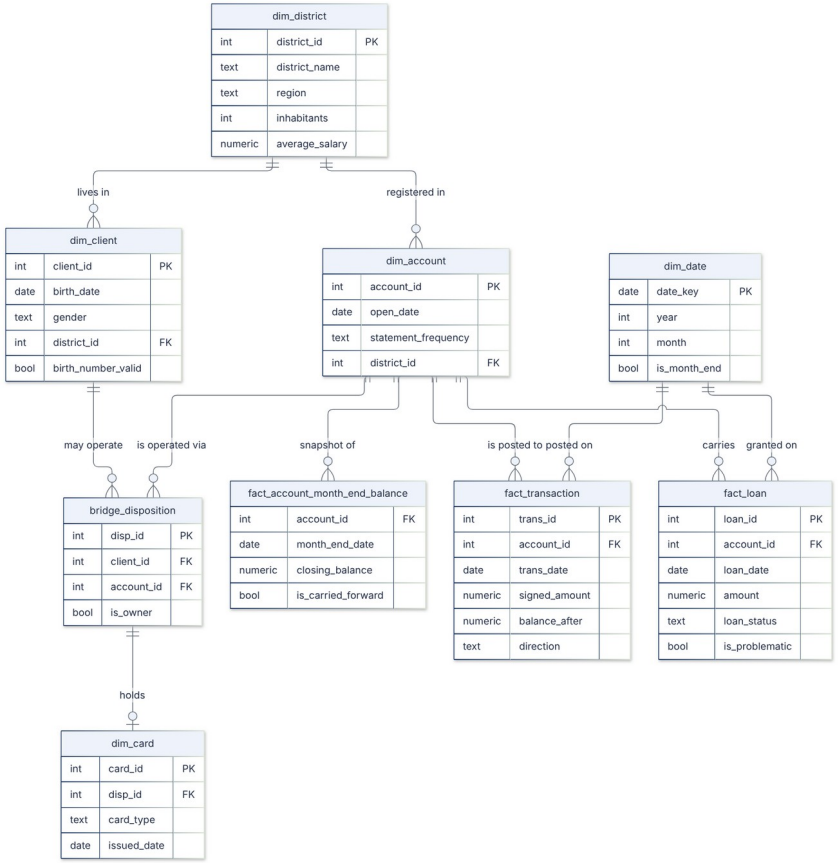
How I know it's true

- 20 of 20 seeded defects caught by the right rule
- raw = stg on 11 checks
- core = stg − quarantine on 9 checks (rows and money)
- Every changed value keeps its original in a _raw column

Proof 2 · One meaning, checked on every build

raw stg dq **core** **ai** sec

you are here



Star schema: 4 grains (client, account, loan, posting), natural keys, verified every build

11 entities, grain and every column described

12 certified metrics with their time column

19 joins with measured cardinality

7 caveats with row counts

Decided once

- Owner = the OWNER disposition
- Problematic = status B or D (76 / 682)
- Incoming = credits, unsigned
- Month-end = last posting, carried forward

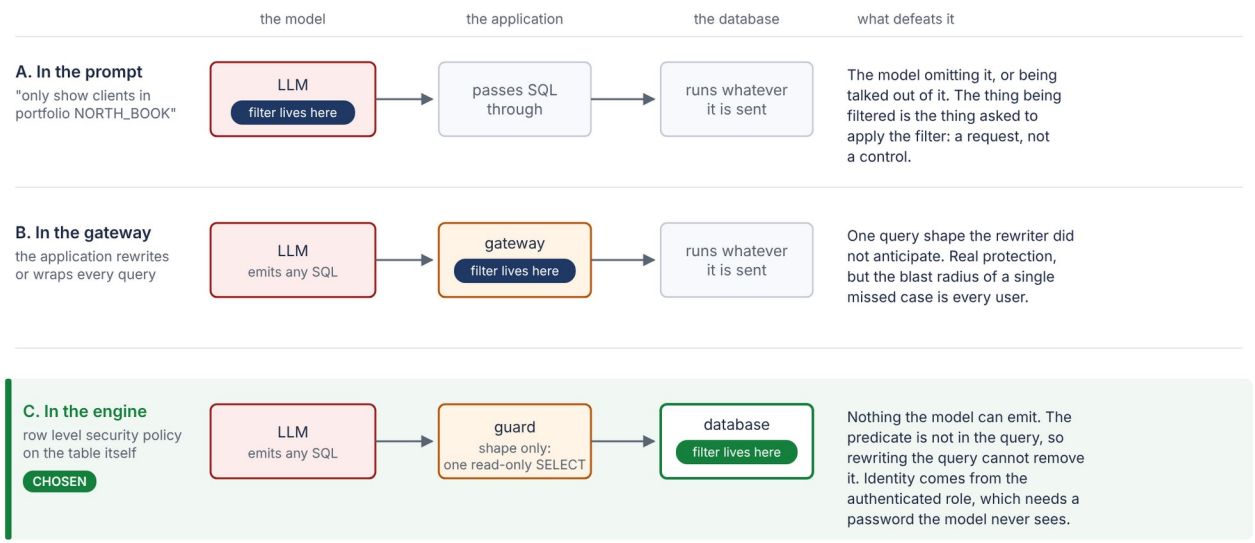
The build fails if …

- a column doesn't exist or isn't described
- a grain isn't one row per key
- a join's cardinality changed
- a metric names a missing column

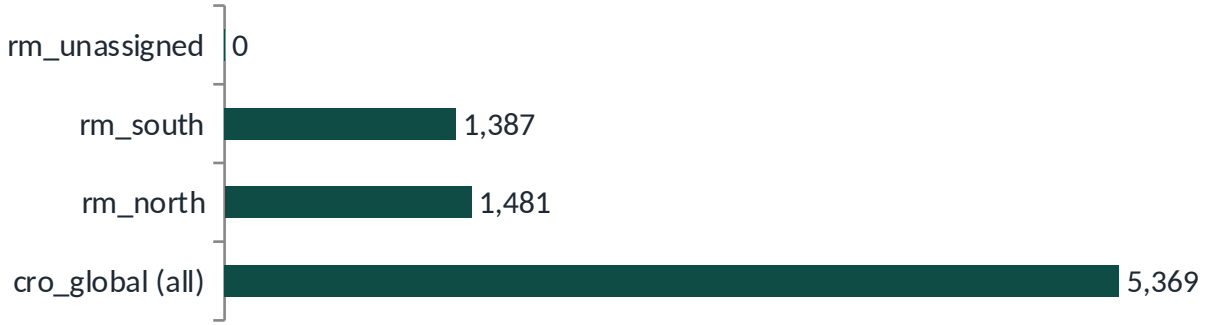
Proof 3 · Access the model cannot bypass

raw stg dq **core** **ai** **sec**

you are here



Clients visible after logging in as each user



Where

7 RLS policies on the client-bearing core tables; every view in between is security_invoker

Why no bypass

The predicate isn't in the model's text; identity is session_user; no BYPASSRLS

Onboarding

Insert a user + portfolio, create the role, enrol TOTP — no code. Offboarding: one UPDATE

Two factors

Password alone → 0 rows; the code is verified inside the database

One question, end to end — and who protects what



Question

"What % of loans are problematic?"

Semantic layer

the model's whole world: no data, no tools

LLM writes SQL

one JSON: sql + reasoning

Guard

one read-only SELECT on ai, LIMIT ≤ 10,000

2FA login

the user's own session, 60 s timeout

RLS on core

only this user's rows exist

Answer + log

rm_north 179 loans · cro_global 76 / 682

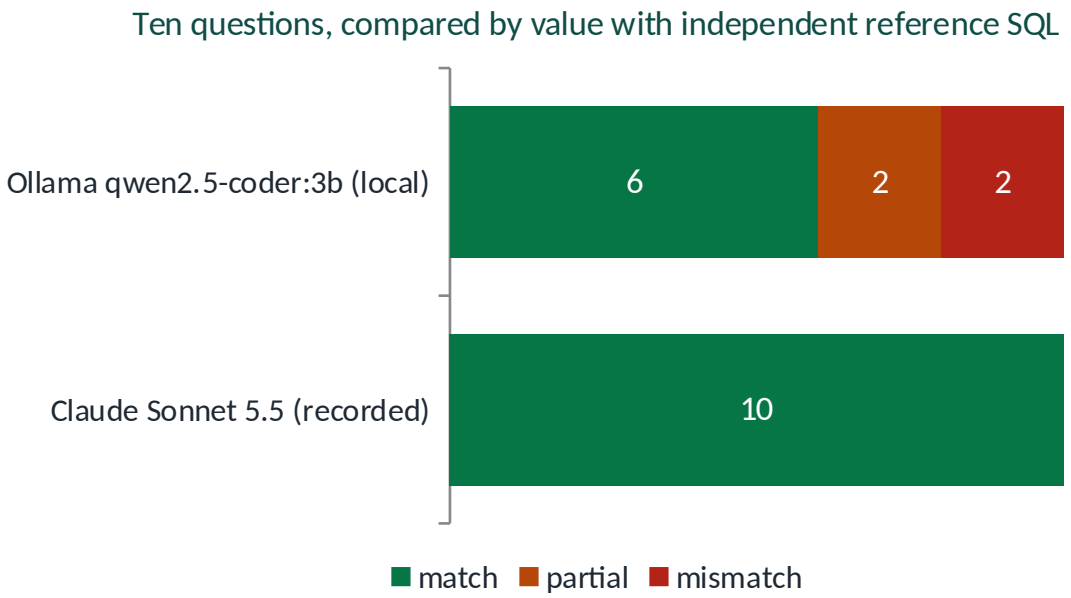
MEANING — removes reasons to guess	SHAPE	ROWS — the security boundary	CHECK

The guard proves a query is safe to run — not that it answers the question.

Observed: "Find male or female loans in Q1 1995" → SQL used T2.gender on ai.account_owner → guard allowed → database: column does not exist. Safe failure, but valid-and-wrong SQL would have returned a plausible number.

Later we fixed it (not in the delivered version): an intent check (D029) refuses this query before it runs.

The evidence — and what it does not prove



10/10
first attempt

~2 min
USD 0.40, every call logged

327
tests passed (1 skipped)

What it shows

- The semantic layer is good enough for a strong model on these questions
- No SQL hand-corrected; references never shown to the model
- A weak model fails in exactly the ways the layer warns about

What it does not show

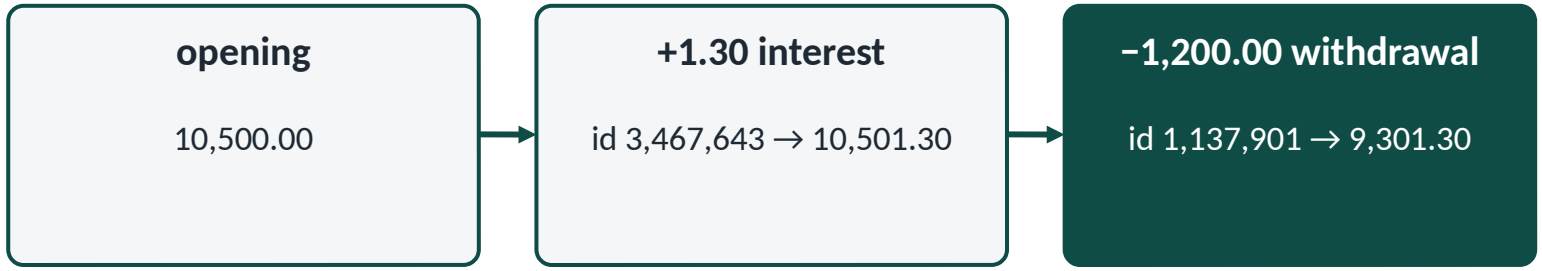
- Generalisation: no hold-out set, and the pack names the questions
- Stability: one sample per question
- Currency: recorded on the pack before the semantic audit

Where I would challenge myself

Month-end tie-break	Highest trans_id picks the closing posting — right 0.4% of the time where the balance chain decides	Disclosed · fixed after
Guard ≠ meaning	A safe query can be wrong; the intent checker was a prototype	Disclosed · fixed after
10/10 is narrow	No hold-out, question labels in the pack, one sample	Partly disclosed
Disponent privacy	Sees the owner's card; a hidden owner shows as 'no card' — false, not hidden	Found after delivery
Scope vs a 1–2 day brief	2FA, web tour, deployment guide, agents — tasks first; I'd cut those first	Talk track

The lesson I'd take to any data team

My best-looking defect — “an account off by exactly 1,200: a missing posting” — was my own bug.



Account 3888, 31 Jan 1998 — each posting starts where the previous one ended. The chain says the withdrawal came last; my rule (highest id) read 10,501.30.

0.4%

how often the highest id was the true closing posting

55 → 14

material reconciliation breaks after the fix

+1.53

change in the Q9 average (43,808.94 → 43,810.47)

107

accounts with a different closing balance

What I learned

- A derived check inherits the bugs of what it reads
- 'Unknowable' is a hypothesis — test it before documenting it
- When a number makes a great story, check it twice

What I did next, and what changes at 100,000×

After delivery — on a separate branch

	Delivered	After
Closing posting	highest trans_id	balance chain (D028)
Q9 average	43,808.94	43,810.47
Material breaks	55	14
Accounts that change	—	107 (max 1,200.00)
Meaning of SQL	guard only	intent verified (D029)
Gender question	DB error	refused before running
Where	fc0c281 (handed in)	this branch, with tests

Not merged: it changes a certified number — that needs the business to sign off the rule.

At 100,000× (≈ 100 bn postings, 10 TB)

- Rebuild-from-scratch becomes incremental: month partitions, merge not replace
- Rules check what arrived; info sampled, block exact
- Month-end maintained from last month + new postings
- Entitlements as a materialised (role, account) set
- A hold-out question set and monitoring

Unchanged: the filter stays in the engine, onboarding stays an insert, the semantic layer stays the same size.



Traceable numbers. No silent repairs. A filter the model cannot reach.

Where it is weak, it is written down — with the evidence and the fix. Questions welcome.

Backup · how the documents fit the story

1–2 Problem & principle

- Brief + questions.csv
- DDR — context, decisions 1 and 7

3 Trustworthy data

- Workbook: Defect types, DQ rules, Cleaning
- dq_rules.yaml · dq_report.md
- CLEANING_LEDGER.md · DDR decisions 2 and 4

3 One meaning

- model_explanation.md · Workbook: Model
- semantic_layer.yaml → context_pack.md
- DDR decisions 3 and 5

3 Access

- ACCESS_AND_AUTH.md · WORKED_EXAMPLE.md
- Workbook: Entitlements
- DDR decision 6

4 Evidence

- Workbook: Questions, Q detail
- llm_results / evaluation.json · llm_logs/
- DELIVERY_AUDIT.md — every claim → source

5 Honesty & next

- Lessons Learned (02) · DECISIONS.md
- ASSUMPTIONS.md · DDR 100,000×
- Study guide (01 / 01b) for depth

Backup · the ten answers, as delivered

Q	Question	Answer
1	Clients per region	south Moravia 937 · north Moravia 920 … (client's own district, all 5,369)
2	Loans granted per year	1993 2.62 m (half year) … 1997 peak 30.73 m · total 103,261,740
3	Share of problematic loans	76 / 682 = 11.14% (status B or D)
4	Average loan by owner's card	junior 221,970 (n = 21) · gold 161,881 · no card 148,865 · classic 148,809
5	Top 10 districts, incoming 1997	Hl.m. Praha 115.13 m · Karvina 28.51 m · Ostrava 27.69 m …
6	New accounts per month 1996	106 · 104 · 124 · 101 · 91 · 113 · 137 · … · 125 (1,363)
7	Owner age at loan start	A 37.07 · B 39.60 · C 38.70 · D 37.00
8	Gold-card owners by gender	male 53 (60.23%) · female 35 (39.77%)
9	Average balance end Dec 1998	43,808.94 over 4,500 accounts (76 carried forward)
10	Highest problematic share by frequency	after transaction 18.75% (6 of 32 — not significant)

Backup · decisions by layer

raw	stg	dq	core	ai	sec
D003 verified public mirror	D013 never repair in place	D008 quarantine scoped to stg	D007 natural keys	D005 sandboxed LLM	D009 filter in the engine
D004 defects only in inject	D014 placeholders → NULL	D006 / D017 one code map, checked	D019 month-end decided once	D010 every view security_invoker	D021 a customer IS a client
	D015 19xx dates, NULL on failure		D012 rebuild restores ai	D011 value-based evaluation	D023 / D024 account, not person
	D016 derived gender, labelled				D025 three identifiers
	D018 sign resolved once				D026 2FA in the database
	D020 graded, not gated				D027 derive grants

Backup · likely questions and where the answer is

If they ask …	Short answer	Go to
Why not DuckDB / a warehouse?	Task 3 needs enforcement in the engine; DuckDB has no RLS	DDR §1
Why never repair a value?	A repaired value looks observed a week later	D013, Proof 1
Your only block is 0.00 CZK?	The rule can only catch zero-amount duplicates; warn rule for batches	L-04
Can the LLM bypass the filter?	It writes text; the predicate isn't in it; identity = session_user	Proof 3
Does passing the guard mean correct?	No — shape only. The gender query proves it	L-07
Is 10/10 meaningful?	Narrow: no hold-out, one sample, older pack	L-08
Biggest mistake?	The tie-break; the 1,200 story was my own artefact	L-01–L-03
Why so much for 1–2 days?	Tasks first; extras requested; cut deployment guide first	L-19

Backup · security residuals, stated before they are asked

Residual	Status	Fix
Direct client rolls back failed second-factor guesses	Disclosed (strict xfail)	Count attempts outside the caller's transaction, or gateway-only logins
Session bound to backend PID only	A45	Compare backend_start too
Disponent sees the owner's card and disposition row	Found after delivery	Holder-based card policy; own-row dispositions; tests
Hidden owner fields shown as 'no card' / NULL	Found after delivery	No coalesce over RLS-hidden joins; say 'not visible'
TOTP secrets recoverable; demo shows codes	A44, A46	Encrypt with an external key; real enrolment
No read audit, no masking, role-per-user pooling	Recommended	Audit log; masking; pool per role or SET ROLE gateway